COMP 018 Midterm Examination - Troubleshooting Plans & Investigation (Owner Role)

Project Under Defense: CampusSafe: Campus Hazard & Safety Reporting System
Defending Group: Group 7 (Jefferson Orpesa Lead, Alba, Arreza, Robiso, Santos)
Assigned Designer Group: Group 12 (iBarangay - Amoranto, Denielle; Eurescial, Eunice; Francisco, Helix)
Target Disposable Database: `campussafe_midterm_test` / `grp07_campussafedb_disposable`
Database Platform: PostgreSQL 18

Step 0: Disposable Database Spin-Up Protocol (RUN FIRST)

In accordance with course rubrics and DBA isolation requirements, all fault simulations, diagnostics, and corrective scripts are executed strictly on an isolated disposable database:

```
# In PowerShell / Command Prompt
createdb -U postgres -h localhost -p 5432 -T grp07_campussafedb grp07_campussafedb_disposable
```

Note: `-T grp07_campussafedb` clones the schema, reference data, and state without touching the primary live database.

Summary of Released Incidents from Designer (Group 12)

Incident ID	Variant & Area	Designer Flaw Description	Owner Result	Status
G07-INC1	Privileges & Access Control (Act 4)	Excessive table-wide UPDATE privileges on app_user	Column-level UPDATE enforced	RESOLVED
G07-INC2	Schema & Referential Integrity (Act 3/4)	ON DELETE CASCADE destroying audit history on delete	FK RESTRICT & soft-delete added	RESOLVED
G07-INC3	Database Triggers & Auditing (Act 3/5)	Missing DB trigger for report_status_history logging	Automated transition trigger built	RESOLVED
G07-INC4	Schema Normalization & Constraints (Act 3/5)	Missing composite unique constraint on campus_location	De-duplicated & UNIQUE added	RESOLVED
G07-INC5	Data Integrity & Check Constraints (Act 3/5)	Nullable priority allowed on 'Verified' hazard reports	Backfilled & conditional CHECK added	RESOLVED

Detailed Investigation, Troubleshooting Plans & Remediation

Incident 1: G07-INC1 (Excessive UPDATE Privileges on Sensitive Columns)

- **Incident ID:** `G07-INC1` (Designer: Group 12)
- **Variant & Activity Area:** Role Privileges & Least-Privilege Access Control (Activity 4)
- **Baseline Reference:** Activity 4 User Roles & Privileges Specification (`docs/user-roles.md`, `database/03_roles_and_privileges.sql`)

1. Observe (Reported Symptoms)

An application account modifies a user's assigned role or account status even though the operation should be restricted to authorized administrators.

2. Verify (Database Inspection & Evidence Gathered)

Inspection of `information_schema.table_privileges` revealed that runtime role `campussafe_app` (member of `app_runtime_grp`) held full table-level `UPDATE` on `app_user`:

```
SELECT grantee, table_name, privilege_type
FROM information_schema.table_privileges
WHERE table_name = 'app_user' AND grantee = 'app_runtime_grp';
```

Observed Output: `grantee: app_runtime_grp | privilege_type: UPDATE` (no column restrictions).

3. Hypothesize (Root Cause Reasoning)

- **Hypothesis A (Confirmed):** Activity 4 database script issued `GRANT UPDATE ON app_user TO app_runtime_grp;`, indiscriminately granting write access to sensitive administrative columns (`role_id`, `account_status`) on `app_user`.
- **Hypothesis B (Ruled Out):** Application authorization bug alone. Ruled out because direct query execution under `campussafe_app` succeeded, confirming the database engine failed to enforce least-privilege defense-in-depth.

4. Planned Corrective Action

- **Target Account Used:** `postgres` (DBA administrative maintenance).
- **Troubleshooting Plan:** Revoke broad table-wide UPDATE privilege from `app_runtime_grp` on `app_user` and replace with granular column-level UPDATE grants on safe user-profile attributes only.
- **Risk Assessment:** Modifying role permissions could break user profile updates if legitimate columns are omitted.
- **Rollback Plan:** `GRANT UPDATE ON app_user TO app_runtime_grp;`

5. Execution of Corrective Action

Executed via `midterm/owner/scripts/fixes/fix_inc1_role_privilege.sql`:

```
-- Step 1: Revoke full-table update
REVOKE UPDATE ON app_user FROM app_runtime_grp;

-- Step 2: Grant column-level UPDATE on safe fields only
GRANT UPDATE (full_name, email, password_hash) ON app_user TO app_runtime_grp;
```

6. Verification & Regression Testing

- **Negative Test (Must Fail):** Executed under role `campussafe_app`: `sql SET ROLE campussafe_app; UPDATE app_user SET account_status = 'suspended' WHERE user_id = 2;` *Result:* `ERROR: permission denied for column "account_status" of relation "app_user" (SQLSTATE 42501)` -> **PASS**.
- **Positive Test (Must Succeed):** Executed under role `campussafe_app`: `sql UPDATE app_user SET full_name = 'Updated User Name' WHERE user_id = 2;` *Result:* `UPDATE 1` -> **PASS**.

7. Root Cause Statement & Classification

- **Confirmed Root Cause:** Table-level UPDATE grant in `database/03_roles_and_privileges.sql` omitted column-level granularity.
- **Responsible Area:** DBA / Access Control Configuration.
- **Status:** `RESOLVED (100%)`.

8. Preventive Recommendation

Establish automated privilege inspection checks in CI/CD pipeline that flag any table-wide `UPDATE` or `DELETE` grants on user credentials and catalog tables.

Incident 2: G07-INC2 (Cascading Deletions Purging Audit & Compliance History)

- **Incident ID:** `G07-INC2` (Designer: Group 12)
- **Variant & Activity Area:** Schema Design, Referential Integrity & Audit Retention (Activity 3 & 4)
- **Baseline Reference:** Activity 3 Database Schema (`database/01_schema.sql`)

1. Observe (Reported Symptoms)

An administrator deletes an old hazard report, causing its associated status history, maintenance records, assignments, and uploaded evidence to disappear from the database.

2. Verify (Database Inspection & Evidence Gathered)

Inspection of foreign key constraints via `pg_constraint` showed `confdeltype = 'c'` (CASCADE) on all child relations referencing `hazard_report`:

```
SELECT conname, conrelid::regclass, confdeltype
FROM pg_constraint
WHERE confrelid = 'hazard_report'::regclass;
```

Observed Output: Foreign keys `fk_history_report`, `fk_maintenance_report`, `fk_assignment_report`, and `fk_attachment_report` all evaluated to `confdeltype: c`.

3. Hypothesize (Root Cause Reasoning)

- **Hypothesis A (Confirmed):** Schema configuration used `ON DELETE CASCADE` on child tables, violating compliance retention requirements for audited hazard reports.
- **Hypothesis B (Ruled Out):** Application trigger deleted child rows explicitly. Ruled out by verifying that no `BEFORE/AFTER DELETE` triggers existed on `hazard_report`.

4. Planned Corrective Action

- **Target Account Used:** `postgres` (DBA administrative maintenance).
- **Troubleshooting Plan:** Alter all child table foreign keys to `ON DELETE RESTRICT` (blocking hard deletion while dependent audit records exist) and add a soft-delete mechanism (`is_deleted`, `deleted_at`) to `hazard_report`.
- **Risk Assessment:** Application queries must filter out soft-deleted records (`WHERE is_deleted = FALSE`).
- **Rollback Plan:** Re-add foreign keys with `ON DELETE CASCADE`.

5. Execution of Corrective Action

Executed via `midterm/owner/scripts/fixes/fix_inc2_cascade_restrict.sql`:

```
-- Step 1: Restrict deletions on dependent audit tables
ALTER TABLE report_status_history DROP CONSTRAINT IF EXISTS fk_status_history_report;
ALTER TABLE report_status_history ADD CONSTRAINT fk_status_history_report
    FOREIGN KEY (report_id) REFERENCES hazard_report(report_id) ON DELETE RESTRICT;

ALTER TABLE maintenance_action DROP CONSTRAINT IF EXISTS fk_maintenance_report;
ALTER TABLE maintenance_action ADD CONSTRAINT fk_maintenance_report
    FOREIGN KEY (report_id) REFERENCES hazard_report(report_id) ON DELETE RESTRICT;

ALTER TABLE assignment DROP CONSTRAINT IF EXISTS fk_assignment_report;
ALTER TABLE assignment ADD CONSTRAINT fk_assignment_report
    FOREIGN KEY (report_id) REFERENCES hazard_report(report_id) ON DELETE RESTRICT;

ALTER TABLE attachment DROP CONSTRAINT IF EXISTS fk_attachment_report;
ALTER TABLE attachment ADD CONSTRAINT fk_attachment_report
    FOREIGN KEY (report_id) REFERENCES hazard_report(report_id) ON DELETE RESTRICT;

-- Step 2: Implement soft deletion on hazard_report
ALTER TABLE hazard_report ADD COLUMN IF NOT EXISTS is_deleted BOOLEAN DEFAULT FALSE;
ALTER TABLE hazard_report ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMPTZ;
```

6. Verification & Regression Testing

- **Negative Test (Must Fail):** Executed under DBA role: `sql DELETE FROM hazard_report WHERE report_id = 1;` *Result:* `ERROR: update or delete on table "hazard_report" violates foreign key constraint "fk_status_history_report" on table "report_status_history" (SQLSTATE 23503)` -> **PASS**.
- **Positive Test (Must Succeed):** Soft delete preserves full history: `sql UPDATE hazard_report SET is_deleted = TRUE, deleted_at = NOW() WHERE report_id = 1; SELECT is_deleted, deleted_at FROM hazard_report WHERE report_id = 1;` *Result:* Report marked deleted while 100% of audit history, attachments, and assignments remain intact -> **PASS**.

7. Root Cause Statement & Classification

- **Confirmed Root Cause:** Overly permissive `CASCADE` referential action in initial schema definition.
- **Responsible Area:** DBA / Schema Architecture.

- **Status:** `RESOLVED (100%)`.

8. Preventive Recommendation

Establish an explicit DBA policy prohibiting `CASCADE` delete rules on any compliance, incident, or financial audit records; mandate soft-deletion patterns.

Incident 3: G07-INC3 (Missing Database Trigger for Status History Auditing)

- **Incident ID:** `G07-INC3` (Designer: Group 12)
- **Variant & Activity Area:** Database Triggers & Automatic Auditing (Activity 3 & 5)
- **Baseline Reference:** Activity 3 Database Documentation (`docs/database.md`)

1. Observe (Reported Symptoms)

An administrator directly changes a hazard report's status through a database management tool, but the corresponding status change does not appear in the report's historical records.

2. Verify (Database Inspection & Evidence Gathered)

Inspection of catalog `pg_trigger` confirmed that no status change triggers were attached to table `hazard_report`:

```
SELECT tgname, tgtype, tgenabled
FROM pg_trigger
WHERE tgrelid = 'hazard_report'::regclass AND NOT tgisinternal;
```

Observed Output: `(0 rows)` -> Status logging was solely handled by application PHP controller logic.

3. Hypothesize (Root Cause Reasoning)

- **Hypothesis A (Confirmed):** State transitions relied on application-level `INSERT` statements inside web controllers rather than an underlying database trigger. Manual updates, batch migrations, or direct queries bypassed history logging.
- **Hypothesis B (Ruled Out):** Trigger was installed but disabled. Ruled out as no trigger definition existed in the database catalog.

4. Planned Corrective Action

- **Target Account Used:** `postgres` (DBA administrative maintenance).
- **Troubleshooting Plan:** Implement an `AFTER UPDATE OF status` trigger on `hazard_report` that automatically writes the previous status, new status, timestamp, and changing user into `report_status_history`.
- **Risk Assessment:** Recursive trigger execution if status is modified within the trigger function. Mitigated by restricting trigger strictly to `AFTER UPDATE OF status` and writing to a different table (`report_status_history`).
- **Rollback Plan:** `DROP TRIGGER trg_hazard_report_status_audit ON hazard_report; DROP FUNCTION trg_log_hazard_status_change();`

5. Execution of Corrective Action

Executed via `midterm/owner/scripts/fixes/fix_inc3_status_trigger.sql`:

```
CREATE OR REPLACE FUNCTION trg_log_hazard_status_change()
RETURNS TRIGGER AS $$
BEGIN
    IF (OLD.status IS DISTINCT FROM NEW.status) THEN
        INSERT INTO report_status_history (
            report_id,
            changed_by_user_id,
            status,
            changed_at,
            notes
        ) VALUES (
            NEW.report_id,
            COALESCE(NEW.verified_by_user_id, 1),
            NEW.status,
            NOW(),
            'Automated database trigger status transition audit: ' || OLD.status || ' -> ' ||
NEW.status
        );
    END IF;
    RETURN NEW;
```

```
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_hazard_report_status_audit ON hazard_report;
CREATE TRIGGER trg_hazard_report_status_audit
AFTER UPDATE OF status ON hazard_report
FOR EACH ROW
EXECUTE FUNCTION trg_log_hazard_status_change();
```

6. Verification & Regression Testing

Test: Executed direct SQL update via psql: `sql UPDATE hazard_report SET status = 'In Progress' WHERE report_id = 1; SELECT report_id, status, changed_at, notes FROM report_status_history WHERE report_id = 1 ORDER BY changed_at DESC LIMIT 1;` *Result:* New audit row automatically populated showing transition -> **PASS**.

7. Root Cause Statement & Classification

Confirmed Root Cause: Relying solely on PHP controllers created a blind spot for direct SQL changes and batch updates.

Responsible Area: DBA / Automation & Auditing.

Status: `RESOLVED (100%)`.

8. Preventive Recommendation

Enforce all core audit trails and compliance state changes at the database engine level via triggers so that external tools or API integrations cannot bypass audit logging.

Incident 4: G07-INC4 (Missing Composite Unique Constraint on Physical Locations)

Incident ID: `G07-INC4` (Designer: Group 12)

Variant & Activity Area: Schema Constraints & Normalization (Activity 3 & 5)

Baseline Reference: Activity 3 Master Data Management (`database/01_schema.sql`, `docs/master-data-crud.md`)

1. Observe (Reported Symptoms)

Administrators encounter multiple entries representing the same building, floor, and room when selecting locations for hazard reports.

2. Verify (Database Inspection & Evidence Gathered)

Inspection of `campus_location` constraints confirmed the table possessed only a surrogate primary key `location_id` and zero composite uniqueness constraints on location dimensions:

```
SELECT conname, contype, pg_get_constraintdef(oid)
FROM pg_constraint
WHERE conrelid = 'campus_location'::regclass;
```

Observed Output: Only `pk_campus_location (PRIMARY KEY (location_id))` was present. Querying duplicates:

```
SELECT building, floor, room_area, COUNT(*)
FROM campus_location
GROUP BY building, floor, room_area HAVING COUNT(*) > 1;
```

Result: Revealed duplicate location entries stored with distinct IDs.

3. Hypothesize (Root Cause Reasoning)

Hypothesis A (Confirmed): Schema design incorporated surrogate primary key `location_id` but omitted the corresponding candidate key constraint on `(building, floor, room_area)`.

Hypothesis B (Ruled Out): Application dropdown bug generated duplicate visual items. Ruled out as duplicate rows physically existed in the database table.

4. Planned Corrective Action

Target Account Used: `postgres` (DBA administrative maintenance).

Troubleshooting Plan: Cleanse existing duplicate rows by updating child references in `hazard_report` to the lowest primary `location_id`, purge duplicate location rows, and add a composite `UNIQUE` constraint.

Risk Assessment: Deleting location rows with existing foreign key references would violate referential integrity. Mitigated by re-pointing references first in Step 1.

Rollback Plan: `ALTER TABLE campus_location DROP CONSTRAINT uq_campus_location_bldg_floor_room;`

5. Execution of Corrective Action

Executed via `midterm/owner/scripts/fixes/fix_inc4_unique_location.sql`:

```
-- Step 1: Re-point foreign keys to primary location ID
WITH ranked_locations AS (
    SELECT location_id,
           FIRST_VALUE(location_id) OVER (PARTITION BY building, floor, room_area ORDER BY
location_id) as keep_id,
           ROW_NUMBER() OVER (PARTITION BY building, floor, room_area ORDER BY location_id) as rn
    FROM campus_location
)
UPDATE hazard_report hr
SET location_id = rl.keep_id
FROM ranked_locations rl
WHERE hr.location_id = rl.location_id AND rl.rn > 1;

-- Step 2: Delete duplicate location rows
WITH duplicates AS (
    SELECT location_id,
           ROW_NUMBER() OVER (PARTITION BY building, floor, room_area ORDER BY location_id) as rn
    FROM campus_location
)
DELETE FROM campus_location
WHERE location_id IN (SELECT location_id FROM duplicates WHERE rn > 1);

-- Step 3: Add composite UNIQUE constraint
ALTER TABLE campus_location
ADD CONSTRAINT uq_campus_location_bldg_floor_room
UNIQUE (building, floor, room_area);
```

6. Verification & Regression Testing

- **Negative Test (Must Fail):** Executed duplicate insertion: `sql INSERT INTO campus_location (building, floor, room_area) VALUES ('Science Hall', '2nd Floor', 'Room 201'); INSERT INTO campus_location (building, floor, room_area) VALUES ('Science Hall', '2nd Floor', 'Room 201');` *Result:* `ERROR: duplicate key value violates unique constraint "uq_campus_location_bldg_floor_room" (SQLSTATE 23505)` -> **PASS**.

7. Root Cause Statement & Classification

- **Confirmed Root Cause:** Reliance solely on surrogate primary key without business natural key constraint.
- **Responsible Area:** DBA / Schema Architecture.
- **Status:** `RESOLVED (100%)`.

8. Preventive Recommendation

Mandate composite natural keys on all dimensional lookup tables to prevent physical record redundancy.

Incident 5: G07-INC5 (Missing Conditional Priority Requirement on Verified Reports)

- **Incident ID:** `G07-INC5` (Designer: Group 12)
- **Variant & Activity Area:** Schema Integrity, Check Constraints & Data Quality (Activity 3 & 5)
- **Baseline Reference:** Activity 3 Hazard Lifecycle Documentation (`docs/database.md`)

1. Observe (Reported Symptoms)

Maintenance personnel encounter verified hazard reports without priority classifications, making it difficult to determine which incidents require immediate attention.

2. Verify (Database Inspection & Evidence Gathered)

Inspection of `hazard_report` column definitions confirmed that `priority` was defined as nullable without conditional check constraints:

```
SELECT column_name, is_nullable, data_type
FROM information_schema.columns
WHERE table_name = 'hazard_report' AND column_name = 'priority';
```

Observed Output: `is_nullable: YES`. Querying verified reports with missing priority:

```
SELECT COUNT(*) FROM hazard_report WHERE status = 'Verified' AND priority IS NULL;
```

Result: Returns verified reports lacking triage priority.

3. Hypothesize (Root Cause Reasoning)

Hypothesis A (Confirmed): Priority was configured as nullable because initial submissions start as `Submitted` without a triage rating. However, the database lacked a conditional check constraint requiring priority once status transitions to `Verified`.

Hypothesis B (Ruled Out): Maintenance view filtered priority incorrectly. Ruled out as the underlying table held NULL priority data.

4. Planned Corrective Action

Target Account Used: `postgres` (DBA administrative maintenance).

Troubleshooting Plan: Backfill existing verified reports holding NULL priority with default `'Medium'`, then add a conditional `CHECK` constraint requiring non-null priority for verified reports.

Risk Assessment: Adding constraint immediately will fail if existing verified rows contain NULL priority. Mitigated by backfilling first in Step 1.

Rollback Plan: `ALTER TABLE hazard_report DROP CONSTRAINT chk_verified_report_requires_priority;`

5. Execution of Corrective Action

Executed via `midterm/owner/scripts/fixes/fix_inc5_priority_check.sql`:

```
-- Step 1: Backfill existing NULLs
UPDATE hazard_report
SET priority = 'Medium'
WHERE status IN ('Verified', 'Assigned', 'In Progress', 'Resolved', 'Closed') AND priority IS
NULL;

-- Step 2: Add conditional CHECK constraint
ALTER TABLE hazard_report
ADD CONSTRAINT chk_verified_report_requires_priority
CHECK (status NOT IN ('Verified', 'Assigned', 'In Progress', 'Resolved', 'Closed') OR priority IS
NOT NULL);
```

6. Verification & Regression Testing

Negative Test (Must Fail): Executed invalid update: `sql UPDATE hazard_report SET status = 'Verified', priority = NULL WHERE report_id = 1;` *Result:* `ERROR: new row for relation "hazard_report" violates check constraint "chk_verified_report_requires_priority" (SQLSTATE 23514)` -> **PASS**.

Positive Test (Must Succeed): `sql UPDATE hazard_report SET status = 'Verified', priority = 'High' WHERE report_id = 1;` *Result:* `UPDATE 1` -> **PASS**.

7. Root Cause Statement & Classification

Confirmed Root Cause: Priority remained nullable across all states without conditional lifecycle validation.

Responsible Area: DBA / Schema Data Integrity.

Status: `RESOLVED (100%)`.

8. Preventive Recommendation

Incorporate state-machine conditional constraints (`CHECK (status != 'X' OR field IS NOT NULL)`) for all lifecycle entities to enforce progressive data completeness.